Project Deliverable for Schema Aware SPARQL Generation

Phase 1: Data and Ontology Preparation

Goal: To establish the "Ground Truth" environment.

1. **Deliverable 1.1: The Domain Ontology (Schema)**
    - Description: A formalized ontology for the specific domain (e.g., in OWL/Turtle/JSON-LD format).
    - Acceptance Criteria: The ontology is valid, loads into a standard editor (like Protege), and covers many classes and sub classes to allow for complex queries.
2. **Deliverable 1.2: The Populated Knowledge Graph (Data)**
    - Description: A graph database populated with instance data related to ontology.
    - Acceptance Criteria: At least 100-500 triples stored in a triple store (or any graph DB) to serve as the target for the generated SPARQL queries.

Phase 2: The "Graph-RAG" Architecture

Goal: To implement the specific retrieval mechanism.

1. **Deliverable 2.1: Ontology Ingestion Pipeline**
    - Description: A Python script that parses the ontology file (from D1.1) and loads it into a Graph Database (like GraphDB, Neo4j).
    - Acceptance Criteria: Classes become nodes, properties become nodes (or relationships), and schema hierarchy is preserved as edges.
2. **Deliverable 2.2: Vector Indexing Module**
    - Description: A system to generate embeddings for each node in the schema graph and store them in a vector index.
    - Acceptance Criteria: Querying the vector index successfully returns the node or property node.

Phase 3: The Retrieval Engine

Goal: To prove the "Subgraph Extraction" concept.

1. **Deliverable 3.1: Seed Node Identification System**
    - Description: Logic that takes a user's prompt, embeds it, and retrieves the top k relevant nodes from the graph.
    - Acceptance Criteria: The system must use the exact same embedding model for the user query as was used to index the graph nodes in Phase 2.

2. **Deliverable 3.2: Subgraph Traversal and Extraction**
    - Description: An algorithm that takes the "seed nodes" and traverses the graph to find the path connecting them using multiple hops.
    - Acceptance Criteria: The system outputs a subgraph representing the relevant schema slice for a given query.
3. **Deliverable 3.3: Subgraph Linearizer**
    - Description: A function that converts the extracted subgraph into a natural language prompt (the "Context").
    - Acceptance Criteria: Output is a clear string that gives the proper context for LLM.

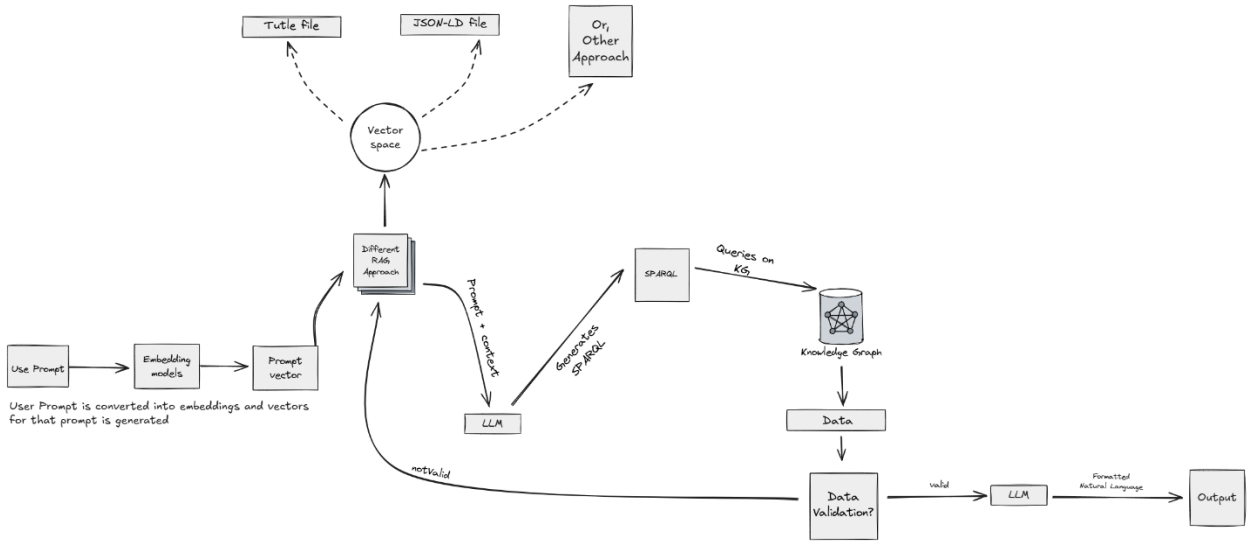
Phase 4: Generation and Integration

Goal: To produce the final SPARQL

1. **Deliverable 4.1: The LLM Integration Module**
    - Description: A Python interface connecting to an LLM (e.g.; GPT-4, Llama-3, or Claude). If feeds the "Verbalized Subgraph" and "User Query" to the model.
    - Acceptance Criteria: The system returns a syntactically valid SPARQL query string.
2. **Deliverable 4.2: End-to-End Pipeline (CLI or UI)**
    - Description: A simple interface where a user inputs a question, and the system runs the SPARQL against the KG (from D1.2) and returns the actual answer.

Phase 5: Evaluation and Analysis

Goal: To scientifically prove this approach works.

1. **Deliverable 4.1: The Benchmark Dataset**
    - Description: A curated list of 30-50 Natural Language questions to generate simple to complex SPARQL queries.
    - Acceptance Criteria: Each question must have a corresponding SPARQL query manually written.
2. **Deliverable 5.2: Evaluation Metrics Report**
    - Description: A comparative analysis using specific metrics:
        - Does the generated query run without error?
        - Does the generated query match the logic of the SPARQL query written manually?
        - Did the RAG system retrieve the right schema nodes?

Basic Workflow Diagram